

Architectural Design

1

1

Software Architecture

The software architecture of a program or computing system is the structure or structures of the system, which comprise software components, the externally visible properties of those components, and the relationships among them.

2

2

Software Architecture Design

- The design process for identifying the sub-systems making up a system and the framework for sub-system control and communication is *architectural design*
- Establishing the overall structure of a software system

3

3

Architectural design process

- **System structuring**
 - The system is decomposed into several principal sub-systems and communications between these sub-systems are identified
- **Modular decomposition**
 - The identified sub-systems are decomposed into modules
- **Control modelling**
 - A model of the control relationships between the different parts of the system is established

4

4

System Structuring

- Concerned with decomposing the system into interacting sub-systems
- The architectural design is normally expressed as a block diagram presenting an overview of the system structure
- More specific models showing how sub-systems share data, are distributed and interface with each other may also be developed

5

5

Architectural styles

- The architectural model of a system may conform to a generic architectural model or style
- An awareness of these styles can simplify the problem of defining system architectures
- However, most large systems are heterogeneous and do not follow a single architectural style

6

6

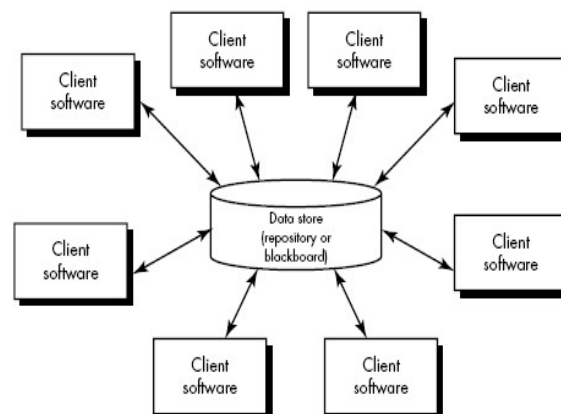
Data-Centered Architecture

- A data store resides at the center of this architecture and is accessed frequently by other components that update, add, delete, or otherwise modify data within the store.
- Existing components can be changed and new client components can be added to the architecture without concern about other clients (because the client components operate independently).

7

7

Data-Centered Architecture



8

8

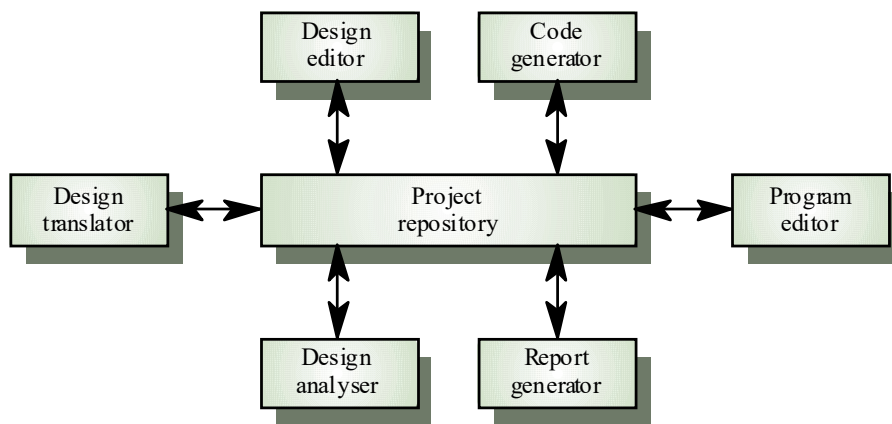
The repository model

- Sub-systems must exchange data. This may be done in two ways:
 - Shared data is held in a central database or repository and may be accessed by all sub-systems
 - Each sub-system maintains its own database and passes data explicitly to other sub-systems
- When large amounts of data are to be shared, the repository model of sharing is most commonly used

9

9

Case tool set architecture



10

10

Repository model characteristics

- **Advantages**
 - Efficient way to share large amounts of data
 - Sub-systems need not be concerned with how data is produced Centralised management e.g. backup, security, etc.
 - Sharing model is published as the repository schema
- **Disadvantages**
 - Sub-systems must agree on a repository data model. Inevitably a compromise
 - Data evolution is difficult and expensive
 - No scope for specific management policies
 - Difficult to distribute efficiently

11

11

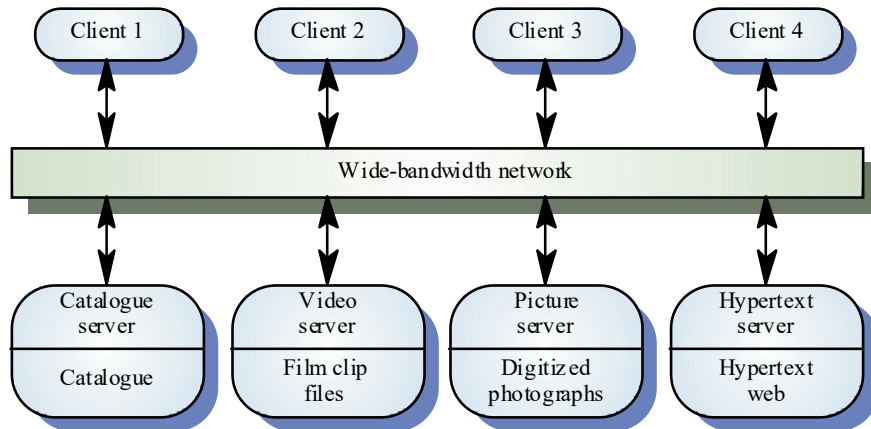
Client-server architecture

- Distributed system model which shows how data and processing is distributed across a range of components
- Set of stand-alone servers which provide specific services such as printing, data management, etc.
- Set of clients which call on these services
- Network which allows clients to access servers

12

12

Film and picture library



13

13

Client-server characteristics

- **Advantages**
 - Distribution of data is straightforward
 - Makes effective use of networked systems. May require cheaper hardware
 - Easy to add new servers or upgrade existing servers
- **Disadvantages**
 - No shared data model so sub-systems use different data organisation. data interchange may be inefficient
 - Redundant management in each server
 - No central register of names and services - it may be hard to find out what servers and services are available

14

14

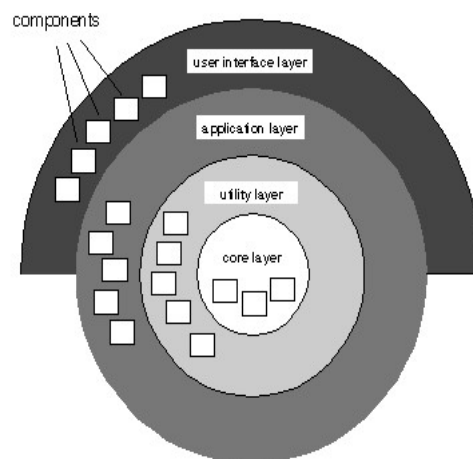
Layered architecture

- A number of different layers are defined, each accomplishing operations that progressively become closer to the machine instruction set.
- At the outer layer, components service user interface operations.
- At the inner layer, components perform operating system interfacing.
- Intermediate layers provide utility services and application software functions.

15

15

Layered Architecture



16

16

Layered architecture characteristics

- **Advantages**
 - The architecture is changeable and portable
- **Disadvantages**
 - Difficult to organize layers
 - Performance may be decreased because of message sending through interface

17

17

Architecture attributes

- **Performance**
 - Localise operations to minimise sub-system communication
- **Security**
 - Use a layered architecture with critical assets in inner layers
- **Safety**
 - Isolate safety-critical components
- **Availability**
 - Include redundant components in the architecture
- **Maintainability**
 - Use fine-grain, self-contained components

18

18

Sub-systems and modules

- A sub-system is a system in its own right whose operation is independent of the services provided by other sub-systems.
- A module is a system component that provides services to other components but would not normally be considered as a separate system

19

19

Modular decomposition

- Another structural level where sub-systems are decomposed into modules
- Two modular decomposition models covered
 - An object model where the system is decomposed into interacting objects
 - A data-flow model where the system is decomposed into functional modules which transform inputs to outputs. Also known as the pipeline model
- If possible, decisions about concurrency should be delayed until modules are implemented

20

20

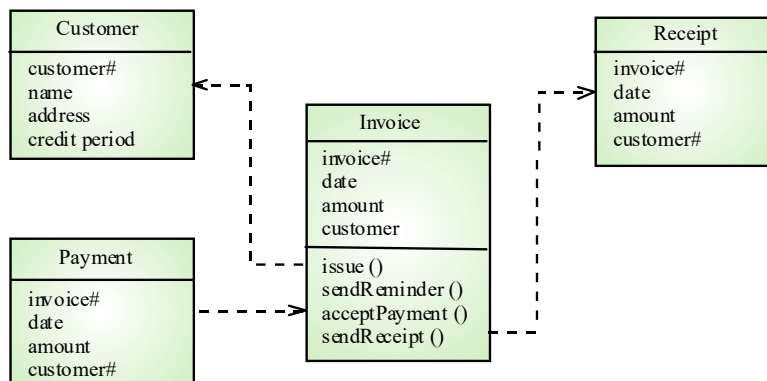
Object models

- Structure the system into a set of loosely coupled objects with well-defined interfaces
- Object-oriented decomposition is concerned with identifying object classes, their attributes and operations
- When implemented, objects are created from these classes and some control model used to coordinate object operations

21

21

Invoice processing system



22

22

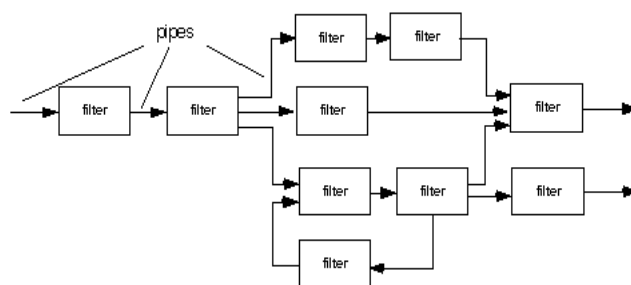
Data-flow architecture

- This architecture is applied when input data are to be transformed through a series of computational or manipulative components into output data.
- A *pipe and filter pattern* has a set of components, called *filters*, connected by pipes that transmit data from one component to the next.
- Each filter works independently of those components upstream and downstream, is designed to expect data input of a certain form, and produces data output (to the next filter) of a specified form
- If the data flow degenerates into a single line of transforms, it is termed *batch sequential*.
- This pattern accepts a batch of data and then applies a series of sequential components (filters) to transform it.

23

23

Data Flow Architecture



(a) pipes and filters

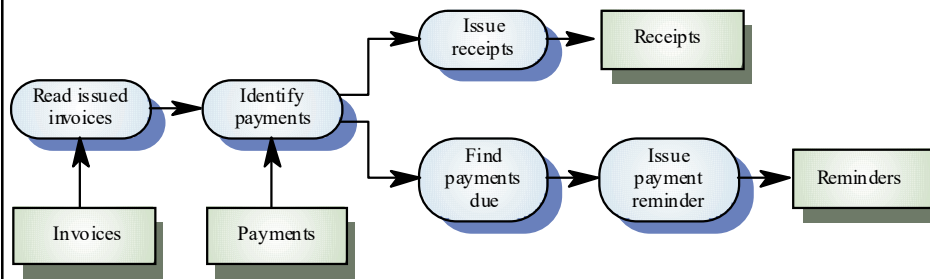


(b) batch sequential

24

24

Data Flow Architecture



25

25

Data-flow architecture characteristics

- **Advantages**

- It supports the reuse of transformations
- It is intuitive in that many people think of their work in terms of input and output processing
- Evolving the system by adding new transformations is usually straightforward.
- It is simple to implement either as a concurrent or a sequential system

- **Disadvantages**

- Not really suitable for interactive systems
- There has to be a common format for data transfer that can be recognised by all transformations.
- Performance may be decreased because of repeated data processing

26

26

Control models

- These models are concerned with the control flow between sub-systems. Distinct from the system decomposition model
- **Centralised control**
 - One sub-system has overall responsibility for control and starts and stops other sub-systems
- **Event-based control**
 - Each sub-system can respond to externally generated events from other sub-systems or the system's environment

27

27

Centralised control

- A control sub-system takes responsibility for managing the execution of other sub-systems
- **Call-return model**
 - Top-down subroutine model where control starts at the top of a subroutine hierarchy and moves downwards. Applicable to sequential systems
- **Manager model**
 - Applicable to concurrent systems. One system component controls the stopping, starting and coordination of other system processes. Can be implemented in sequential systems as a case statement

28

28

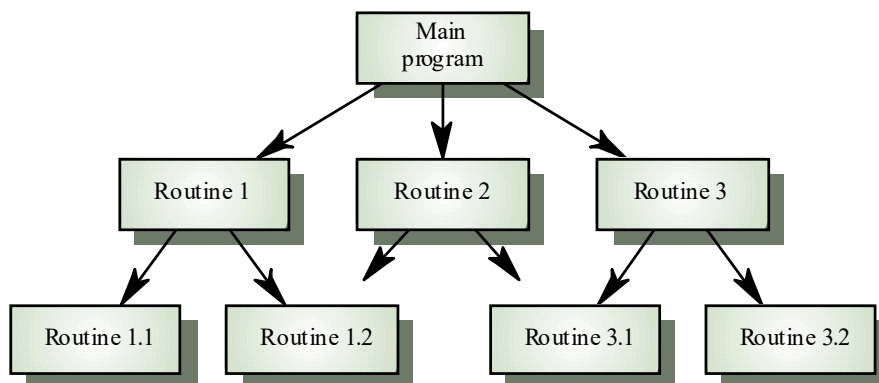
Call and Return Architecture

- *Main program/subprogram architectures*. This classic program structure decomposes function into a control hierarchy where a “main” program invokes a number of program components, which in turn may invoke still other components.
- *Remote procedure call architectures*. The components of a main program/ subprogram architecture are distributed across multiple computers on a network

29

29

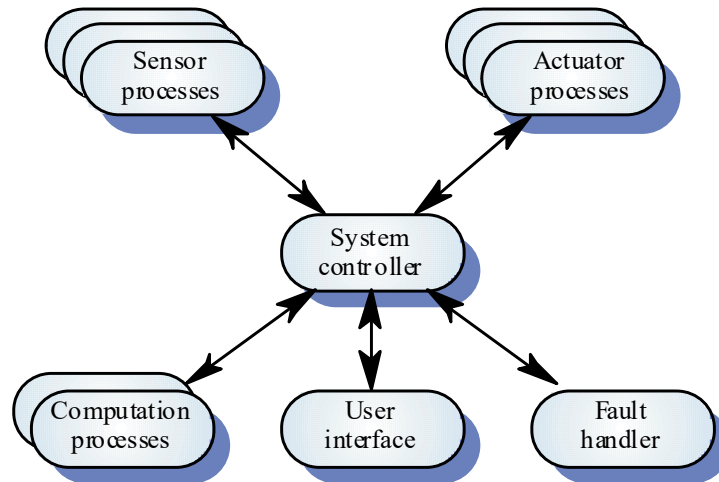
Call and Return Architecture



30

30

Real-time system control



31

31

Event-driven systems

- Driven by externally generated events where the timing of the event is out with the control of the sub-systems which process the event
- Two principal event-driven models
 - Broadcast models. An event is broadcast to all sub-systems. Any sub-system which can handle the event may do so
 - Interrupt-driven models. Used in real-time systems where interrupts are detected by an interrupt handler and passed to some other component for processing

32

32

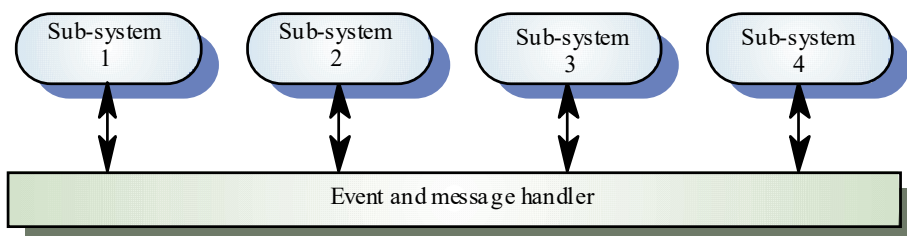
Broadcast Model

- Effective in integrating sub-systems on different computers in a network
- Sub-systems register an interest in specific events. When these occur, control is transferred to the sub-system which can handle the event
- Control policy is not embedded in the event and message handler. Sub-systems decide on events of interest to them
- However, sub-systems don't know if or when an event will be handled

33

33

Selective broadcasting



34

34

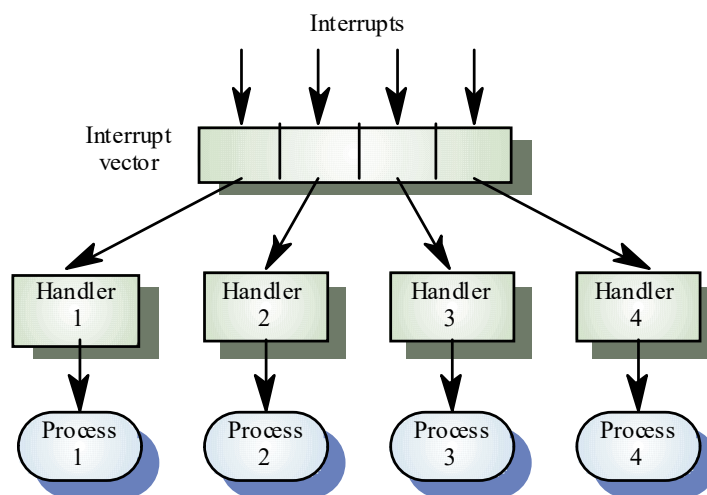
Interrupt-driven systems

- Used in real-time systems where fast response to an event is essential
- There are known interrupt types with a handler defined for each type
- Each type is associated with a memory location and a hardware switch causes transfer to its handler
- Allows fast response but complex to program and difficult to validate

35

35

Interrupt-driven control



36

36